

Clawbox Security Deep Dive

Containerized AI Agent Platform — Architecture & Security Analysis

April 3, 2026

INTERNAL — PREPARED FOR TECHNICAL REVIEW

Executive Summary

Clawbox is a Docker-based deployment kit that runs an AI agent ([OpenClaw](#)) inside an isolated container. It enables software engineers to interact with an LLM-powered coding agent via a CLI, while enforcing security boundaries that prevent the agent from accessing the broader network, host filesystem, or credentials beyond what's explicitly granted.

This document covers the security architecture, the specific controls implemented, validated test results, and the remaining known gaps with their risk assessments.

Bottom line: Clawbox follows defense-in-depth principles — network isolation, filesystem restrictions, capability dropping, resource limits, and credential separation are layered so that no single control failure results in full compromise. The most significant remaining gap is that the exec tool's child processes don't yet inherit virtual memory limits (ISSUE-18), and the egress proxy allowlist can be bypassed if an attacker finds a way to make raw TCP connections that skip the HTTP CONNECT proxy.

What Is Clawbox?

Clawbox packages an OpenClaw AI agent gateway inside a Docker container. A developer on your team runs `bash setup.sh`, provides an Anthropic API key, and gets a working agent they can talk to via:

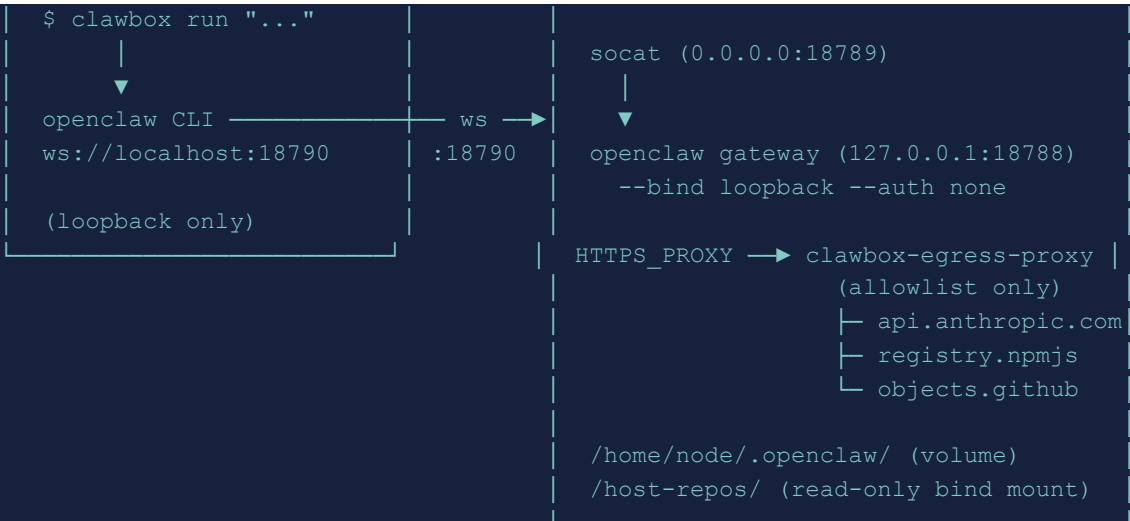
```
clawbox run "review this PR and suggest improvements"
clawbox chat # interactive session
```

The agent has access to a **coding** tool profile — it can read/write files, execute shell commands, and search the web (optionally) within its container. It **cannot** access the host filesystem, push to GitHub, send emails, or reach arbitrary internet hosts.

Architecture

Host Machine

Docker: clawbox-work
(read-only rootfs, non-root user)



Network Topology

Network	Type	Connected Services	Purpose
clawbox-internal	Docker internal: true	clawbox-work	No external routing — containers here cannot reach the internet directly
clawbox-egress	Docker bridge	clawbox-work, clawbox-egress-proxy	Agent → proxy only. Proxy has internet access; agent does not.

The agent container sits on **both** networks. The `internal` network has no default gateway, so the container cannot route to the internet. The `egress` network connects it only to the proxy, which enforces an allowlist before forwarding HTTPS CONNECT requests to the real internet.

Security Controls — Layer by Layer

1. Network Isolation & Egress Proxy

How it works

All outbound HTTPS from the agent is routed through a dedicated proxy container (`clawbox-egress-proxy`) via `HTTPS_PROXY` and `HTTP_PROXY` environment variables. The proxy implements an HTTP CONNECT allowlist:

Allowed Host	Reason
<code>api.anthropic.com</code>	LLM API calls (core functionality)
<code>registry.npmjs.org</code>	Package installation
<code>objects.githubusercontent.com</code>	GitHub raw file downloads
<code>*.npm.org</code>	npm metadata

Everything else — `google.com` , `evil.com` , arbitrary IPs — returns `403 Forbidden` and the connection is destroyed.

Node.js native `fetch()` (used by the Anthropic SDK) requires special handling because it uses `undici` internally, which doesn't respect `HTTPS_PROXY` . This is solved by injecting `proxy-bootstrap.js` via `NODE_OPTIONS=--require` , which calls `setGlobalDispatcher(new ProxyAgent(...))` at process startup.

Validated by

Tegress isolation test — 11/12 pass (1 warn: transient DNS). Confirmed: `example.com BLOCKED`, `google.com BLOCKED`, `evil.com BLOCKED`, `api.anthropic.com ALLOWED`, `npm install ALLOWED`.

2. Read-Only Root Filesystem

How it works

`read_only: true` in `docker-compose.yml` makes the entire container rootfs read-only at the kernel level. The agent cannot modify:

- System binaries (`/usr/bin` , `/sbin`)
- Libraries (`/lib` , `/usr/lib`)
- Configuration (`/etc`)
- The Node.js runtime or npm global packages

Writable paths are explicitly defined:

Path	Type	Size	Purpose

<code>/tmp</code>	tmpfs	256 MB	Ephemeral scratch space (cleared on restart)
<code>/run</code>	tmpfs	32 MB	Runtime state
<code>/home/node/.openclaw</code>	Docker volume	Persistent	Workspace, configs, memory, cache

Redirections ensure that tools expecting writable paths still work:

- `GIT_CONFIG_GLOBAL=/home/node/.openclaw/.gitconfig` → git config writes to volume
- `NPM_CONFIG_CACHE=/home/node/.openclaw/.npm-cache` → npm cache to volume

Validated by

T13 — read-only rootfs test: 12/12 pass. Confirmed write rejection at `/`, `/usr`, `/bin`, `/lib`, `/etc`, `/sbin`. Confirmed `/tmp`, `/run`, and volume are writable. Confirmed git and npm work through redirections. Agent functional test passed.

3. Capability Dropping & Privilege Restrictions

```
cap_drop:
  - ALL
cap_add:
  - CHOWN
  - SETUID
  - SETGID
  - DAC_OVERRIDE
security_opt:
  - no-new-privileges:true
```

ALL Linux capabilities are dropped, then only the four needed for normal container operation are re-added. `no-new-privileges` prevents any process from gaining additional privileges via setuid binaries or capability escalation. Additionally, all setuid/setgid bits are stripped from binaries in the Dockerfile build step.

4. Resource Limits

Limit	Value	Purpose
-------	-------	---------

Memory	10 GB	Prevent OOM on host from runaway agent
CPU	1.0 core	Prevent CPU monopolization
PIDs (cgroup)	512	Prevent fork bombs
nproc (ulimit)	256 soft / 512 hard	Second layer PID limit
nofile (ulimit)	1024 soft / 4096 hard	Prevent file descriptor exhaustion

5. Non-Root Execution

The container runs as user `node` (uid 1000). The Dockerfile switches to this user via `USER node` before the entrypoint. Combined with `no-new-privileges` and stripped setuid bits, there is no path to root inside the container.

6. Credential Isolation

Credential	Access	Rationale
Anthropic API Key	✓ Available (env var)	Core functionality — LLM calls
GitHub Token	✗ Not available	Prevents prompt injection → repo access. Host handles git operations.
Host SSH Keys	✗ Not available	No host filesystem access
Host .env / secrets	✗ Not available	.env file is mode 600, not mounted into container

Design decision — no GitHub credentials in container: The agent can read host repositories via a read-only bind mount (`/host-repos:ro`) but cannot push, create repos, or authenticate to GitHub. This is an intentional security boundary: if a malicious prompt tricks the agent into exfiltrating code, the blast radius is limited to what's already visible in the read-only mount. The agent cannot create repos, modify remote branches, or access private repos it hasn't been explicitly given.

7. Gateway Authentication

The OpenClaw gateway runs with `--auth none` and `--bind loopback` (port 18788 inside the container). The socat proxy bridges to port 18789. Docker maps `127.0.0.1:18790 → container:18789`.

This means the gateway is **only accessible from the host machine's loopback interface**. No other device on the local network can reach it. The `auth=none` eliminates token management friction — acceptable because the port is never network-exposed.

8. Web Browsing Controls

By default, `web_search`, `web_fetch`, and `browser` tools are denied via `tools.deny`. The agent cannot browse the internet even if the proxy allowed it. Set `OPENCLAW_ALLOW_WEB=1` to re-enable (not recommended for untrusted workloads).

Validated Test Results

Test	Coverage	Result	Last Run
T-egress	Network isolation, proxy allowlist, exfiltration simulation	11/12 pass	2026-04-01
T13	Read-only rootfs, tmpfs mounts, volume redirects	12/12 pass	2026-04-01
T8	Context injection, input validation	10/10 pass	2026-04-02
T20	Workspace persistence across restarts	13/13 pass	2026-04-03
T1	Long-running task stability	Pass (30/30 tests)	2026-04-03
T4	Error recovery	Pass	2026-04-02
T5	Real-world onboarding	Pass	2026-04-02
T6	Concurrent tasks	Pass	2026-04-02
T15	Task timeout + handoff	13/13 pass	2026-04-03

Known Gaps & Remaining Risks

Gap	Risk	Impact	Mitigation Status
-----	------	--------	-------------------

ISSUE-18: exec-tool child processes don't inherit virtual memory limits	Medium	A rogue script spawned by the agent could allocate excessive virtual memory. The cgroup memory limit (10 GB) still caps RSS, but VIRT can grow unbounded.	Documented. Requires gateway support for per-exec-child ulimit propagation. Cgroup memory limit provides a hard backstop.
Raw TCP bypass: If agent code makes raw TCP connections (not HTTP CONNECT), traffic bypasses the proxy	Medium	The <code>internal: true</code> Docker network blocks direct internet routing, but if the agent can resolve DNS and construct raw TCP to the proxy's egress network, it could potentially tunnel traffic.	Mitigated by Docker's internal network having no default gateway. The agent cannot route to the internet without going through the HTTP CONNECT proxy. A determined attacker could potentially use the proxy connection itself as a tunnel, but this requires significant sophistication.
Anthropic API key exposure: Key is available as an env var inside the container	Low	If the agent is tricked into printing <code>\$ANTHROPIC_API_KEY</code> , the key is exposed to whoever reads the output. The key cannot be exfiltrated to an external server (blocked by egress proxy).	Accepted risk. The agent needs the key to function. Env var exposure is standard for containerized apps. Key rotation is recommended.
npm package supply chain: Agent can install arbitrary npm packages	Medium	A malicious npm package could execute arbitrary code inside the container during install. Blast radius is limited by read-only rootfs, no GitHub creds, and egress proxy.	Partially mitigated. The read-only rootfs prevents persistent modification. Egress proxy prevents data exfiltration to non-allowlisted hosts. Consider adding a package allowlist or disabling npm in high-security deployments.
Prompt injection via host-repos: Malicious code in read-only mounted repos could influence agent behavior	Low	If a mounted repo contains adversarial content (e.g., a README with hidden instructions), the agent might follow those instructions. However, the agent cannot push changes, access credentials, or reach external hosts.	Mitigated by defense-in-depth. Even if the agent is influenced, it lacks the capabilities to cause significant damage outside the container.
Container escape: Kernel/Docker	Low	A zero-day container escape would give the agent host-level access.	Standard industry risk. Mitigated by running as non-

vulnerability			root, dropping all capabilities, no-new-privileges, and keeping Docker/kernel updated. Consider gVisor or Kata Containers for defense-in-depth if running untrusted workloads.
---------------	--	--	--

Recommendations

For Production Deployment

1. **Rotate API keys regularly** — the Anthropic key is the highest-value credential in the system.
2. **Pin the OpenClaw version** in the Dockerfile rather than installing `latest`.
3. **Add npm package allowlisting** if the agent will run in environments with sensitive data.
4. **Consider gVisor/Kata** for an additional container isolation layer if running untrusted prompts.
5. **Monitor proxy logs** — the egress proxy logs every ALLOWED and BLOCKED connection.
6. **Scope host-repo mounts** — only mount the specific repos the agent needs, not a parent directory.

For Evaluation

1. Run `bash tests/T-egress-isolation.sh` to validate network isolation on your own infrastructure.
2. Run `bash tests/T13-readonly-rootfs.sh` to validate filesystem restrictions.
3. Review `proxy/proxy.js` (48 lines) — the entire egress allowlist is trivially auditable.
4. Review `docker-compose.yml` — all security controls are declared in one file.

Summary of Defense-in-Depth Layers

Layer	Control	Prevents
1	Docker internal network	Direct internet access
2	Egress proxy allowlist	Connections to non-essential hosts
3	Read-only rootfs	Persistent system modification

4	Non-root + no-new-privileges	Privilege escalation
5	Capability drop (ALL)	Kernel-level privilege abuse
6	Resource limits (mem/cpu/pids)	Denial of service on host
7	No GitHub credentials	Code exfiltration / repo manipulation
8	Loopback-only gateway port	Network-adjacent attacks
9	Web tool deny list	Arbitrary web browsing
10	Host repos read-only mount	Source code modification

Contact: Questions about this deployment can be directed to Xander Hunt (xander@paramify.com), who maintains the Clawbox project and the host-side OpenClaw instance that manages it.